Отчёт по лабораторной работе №4

Инструменты воспроизводимых исследований. DrWatson

Федорова А. И.

Содержание

1 Цель работы **5**

2 Задание **6**

3 Теоретическое введение **7**

4 Выполнение лабораторной работы **9**

4.1 Установка пакетов и компиляция зависимостей 9
4.2 Проверка функций навигации DrWatson 9
4.3 Сохранение результатов симуляции 10
4.4 Пакетная обработка параметров через dict_list 11
4.5 Сбор результатов в DataFrame 12
4.6 Создание Makefile для автоматизации 13
4.7 Создание нового проекта через PkgTemplates 14

5 Выводы **16**

Список литературы **17**

Список иллюстраций

4.1 Завершение компиляции зависимостей проекта MyProject 9
4.2 Проверка функций навигации и ошибка при вызове scriptdir . . . 10
4.3 Сохранение результатов симуляции с помощью savename и safesave 11
4.4 Ошибка при запуске цикла симуляций — функция simulate не определена 12
4.5 Сбор результатов в DataFrame и ошибка при выборе столбцов параметров 13
4.6 Makefile для автоматизации сборки проекта 14
4.7 Скрипт PkgTemplates.jl для создания нового проекта с DrWatsonPlugin 15

Список таблиц

3.1 Основные функции пакета DrWatson 7

1 Цель работы

Освоить инструменты для организации воспроизводимых научных исследований на языке Julia с использованием пакета DrWatson. Изучить структуру научного проекта, функции навигации, сохранения и сбора результатов, а также автоматизацию сборки отчёта.

2 Задание

1. Установить пакет DrWatson и инициализировать проект `MyProject`.
2. Изучить стандартные функции навигации по проекту: `datadir()`, `plotsdir()`, `projectdir()`, `scriptsdir()`.
3. Сохранить результаты симуляции с помощью `savename`, `@dict`, `safesave`.
4. Выполнить пакетную обработку параметров через `dict_list` и цикл симуляций.
5. Собрать результаты из файлов в DataFrame с помощью `collect_results`.
6. Создать Makefile для автоматизации запуска скриптов и рендеринга отчёта.
7. Создать шаблон нового проекта через `PkgTemplates.jl` с плагином `DrWatsonPlugin`.

3 Теоретическое введение

DrWatson — пакет Julia, разработанный специально для организации воспроизводимых научных проектов. Он предоставляет соглашения об именовании файлов, навигацию по структуре проекта и утилиты для управления данными.

Стандартная структура проекта, создаваемая DrWatson:

```
MyProject/
├── Project.toml        # Описание зависимостей
├── Manifest.toml       # Точные версии пакетов
├── data/
│   ├── sims/           # Результаты симуляций
│   ├── exp_raw/        # Сырые экспериментальные данные
│   └── exp_pro/        # Обработанные данные
├── scripts/            # Скрипты для анализа
├── src/                # Исходный код проекта
└── _research/          # Черновики и временные файлы
```

Таблица 3.1: Основные функции пакета DrWatson

Функция	Описание
`datadir()`	Путь к директории `data/`
`plotsdir()`	Путь к директории `plots/`
`projectdir()`	Корень проекта

Функция	Описание
`srcdir()`	Путь к директории `src/`
`savename(params, ext)`	Имя файла по словарю параметров
`safesave(path, data)`	Сохранение без перезаписи
`collect_results(dir)`	Сбор результатов в DataFrame
`dict_list(d)`	Декартово произведение параметров
`@dict(a=1, b=2)`	Создание словаря с именованными ключами
`@quickactivate`	Автоматическая активация окружения

PkgTemplates.jl позволяет генерировать стандартную структуру Julia-проекта с нужными плагинами, в том числе `DrWatsonPlugin()`.

Makefile используется для автоматизации запуска скриптов и рендеринга финального документа через Quarto.

4 Выполнение лабораторной работы

4.1 Установка пакетов и компиляция зависимостей

Сначала был установлен пакет DrWatson и инициализирован проект `MyProject`. В процессе Julia скомпилировала 2 новых зависимости — `nghttp2_jll v1.64.0+1` и `p7zip_jll v17.7.0+0`, а также подтвердила компиляцию пакетов JLD2. Система сообщила, что 43 пакета уже были предкомпилированы, и рекомендовала перезапустить Julia для применения новых версий (рис. 4.1).

```
[8e850ede] + nghttp2_jll v1.64.0+1
[3f19e933] + p7zip_jll v17.7.0+0
✓ JLD2
✓ JLD2 → UnPackExt
recompiling packages finished.
2 dependencies successfully precompiled in 94 seconds. 43 already precompiled.
2 dependencies precompiled but different versions are currently loaded. Restart julia to access the new
versions. Otherwise, loading dependents of these packages may trigger further precompilation to work wit
the unexpected versions.
MyProject"
```

Рисунок 4.1: Завершение компиляции зависимостей проекта MyProject

4.2 Проверка функций навигации DrWatson

После активации проекта были проверены функции навигации. Вызовы `datadir()`, `plotsdir()` и `projectdir()` корректно вернули абсолютные пути к соответствующим директориям внутри `MyProject` (рис. 4.2).

При попытке вызвать `scriptdir("analysis.jl")` возникла ошибка:

```
ERROR: UndefVarError: `scriptdir` not defined in `Main`
```

Это ожидаемо: в DrWatson нет функции `scriptdir` — правильное название `scriptsdir()` (с буквой **s** в конце, по аналогии с остальными функциями).

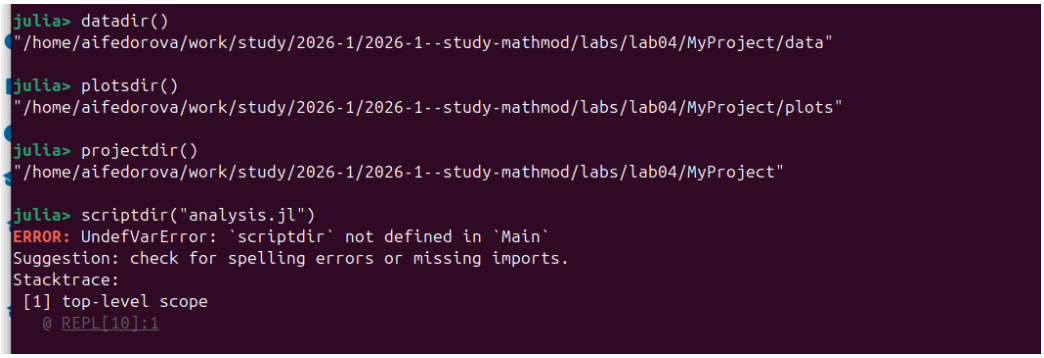

Рисунок 4.2: Проверка функций навигации и ошибка при вызове scriptdir

4.3 Сохранение результатов симуляции

Были импортированы `savename`, `@dict`, `safesave` из DrWatson. Создан словарь параметров и словарь результатов с массивом из 10 случайных чисел:

```
params = @dict(□=0.1, □=2.0)
results = Dict(:data => rand(10))
```

Функция `savename(params, "jld2")` автоматически сформировала имя файла "□=0.1_□=2.0.jld2". Вызов `safesave(datadir("sims", filename), results)` сохранил данные, выведя предупреждение о том, что символьный ключ `:data` был автоматически конвертирован в строку "data" (рис. 4.3).

```
julia> using DrWatson: savename, @dict, safesave

julia> params = @dict(α=0.1, β=2.0)
Dict{Symbol, Float64} with 2 entries:
  :α => 0.1
  :β => 2.0

julia> results = Dict(:data => rand(10))
Dict{Symbol, Vector{Float64}} with 1 entry:
  :data => [0.894939, 0.35093, 0.0206195, 0.0473639, 0.880771, 0.0660871, 0.752385, 0.521048, 0.0145327,…

julia> filename = savename(params, "jld2")
"α=0.1_β=2.0.jld2"

julia> safesave(datadir("sims", filename), results)
┌ Warning: you passed a key as a symbol instead of a string, it has been automatically converted
└ @ JLD2 ~/.julia/packages/JLD2/hbsZG/src/fileio.jl:10
```

Рисунок 4.3: Сохранение результатов симуляции с помощью savename и safesave

4.4 Пакетная обработка параметров через dict_list

Был подключён пакет `BSON` и задан словарь всех параметров:

```
all_params = Dict(:□ => [0.1, 0.5], :□ => 1:5)
```

Функция `dict_list(all_params)` раскрывает это в 10 комбинаций параметров (декартово произведение). Для каждой комбинации планировался вызов пользовательской функции `simulate` и сохранение результата через `safesave`.

Однако при запуске цикла возникла ошибка (рис. 4.4):

```
ERROR: UndefVarError: `simulate` not defined in `Main`
```

Функция `simulate` не была определена в текущей сессии. Согласно рекомендациям DrWatson, пользовательские функции следует выносить в модуль `src/MyProject.jl` и подключать через `using MyProject`.

```
  1 dependency successfully precompiled in 5 seconds

julia> using BSON

julia> all_params = Dict(:α => [0.1, 0.5], :β => 1:5)
Dict{Symbol, AbstractVector} with 2 entries:
  :α => [0.1, 0.5]
  :β => 1:5

julia> for params in dict_list(all_params)
           result = simulate(params)
           safesave(datadir("sims", savename(params, "bson")), result)
       end
ERROR: UndefVarError: `simulate` not defined in `Main`
Suggestion: check for spelling errors or missing imports.
Stacktrace:
 [1] top-level scope
   @ ./REPL[24]:2
```

Рисунок 4.4: Ошибка при запуске цикла симуляций — функция simulate не определена

4.5 Сбор результатов в DataFrame

После подключения `DataFrames` была вызвана функция `collect_results(datadir("sims"))`. DrWatson просканировал папку `data/sims`, обнаружил 1 файл и добавил запись в DataFrame. Итоговая таблица содержала столбцы `data` (массив значений) и `path` (путь к файлу) (рис. 4.5).

При попытке выбрать столбцы `:α`, `:β`, `:result` через `select!` получена ошибка:

```
ArgumentError: column name :α not found in the data frame
```

Параметры α и β не попали в DataFrame, поскольку при сохранении через `safesave` они не были включены в словарь результатов. Для корректного сбора параметров следует использовать `tagsave` вместо `safesave`, либо объединять параметры с результатами перед сохранением: `safesave(path, merge(params, results))`.

Рисунок 4.5: Сбор результатов в DataFrame и ошибка при выборе столбцов параметров

4.6 Создание Makefile для автоматизации

Был создан файл `Makefile` в корне проекта (рис. 4.6):

```
.PHONY: report clean


report:
    julia --project=. scripts/run_simulations.jl
    julia --project=. scripts/generate_report.jl
    quarto render docs/report.qmd


clean:
    rm -rf data/sims/*
    rm -rf plots/*
```

Цель `report` последовательно выполняет запуск симуляций, генерацию про-

межуточных файлов и рендеринг финального документа через Quarto. Цель `clean` очищает директории с результатами и графиками.

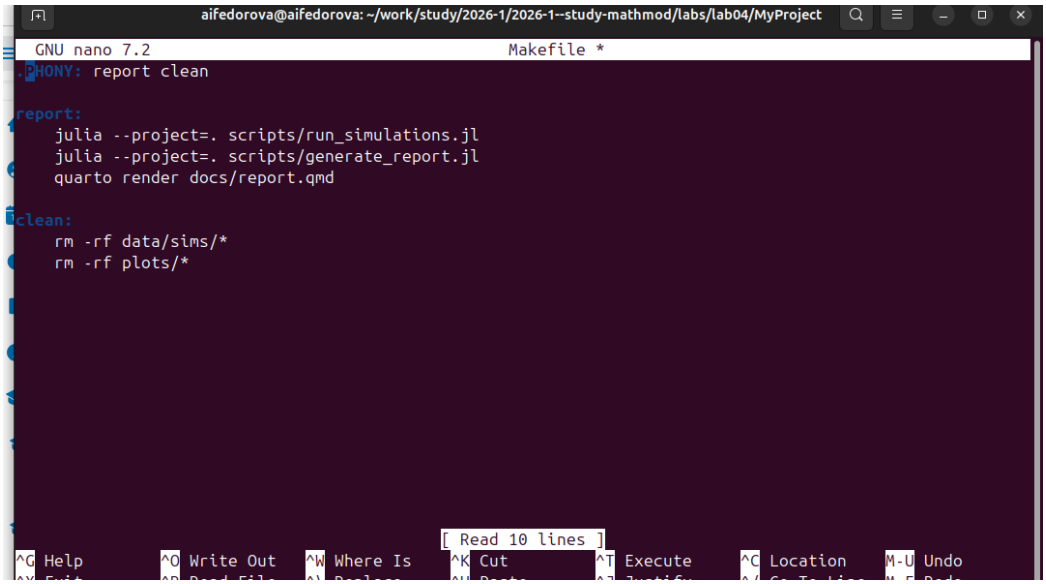

Рисунок 4.6: Makefile для автоматизации сборки проекта

4.7 Создание нового проекта через PkgTemplates

Был создан скрипт `scripts/PkgTemplates.jl` для генерации нового проекта по шаблону с интеграцией DrWatson (рис. 4.7):

```
using PkgTemplates
t = Template(;
    dir="~/Projects",
    julia=v"1.9",
    plugins=[DrWatsonPlugin()]
)
t("MyNewProject")
```

Скрипт создаёт проект `MyNewProject` в директории `~/Projects` со стандартной структурой DrWatson и настройками для Julia 1.9.

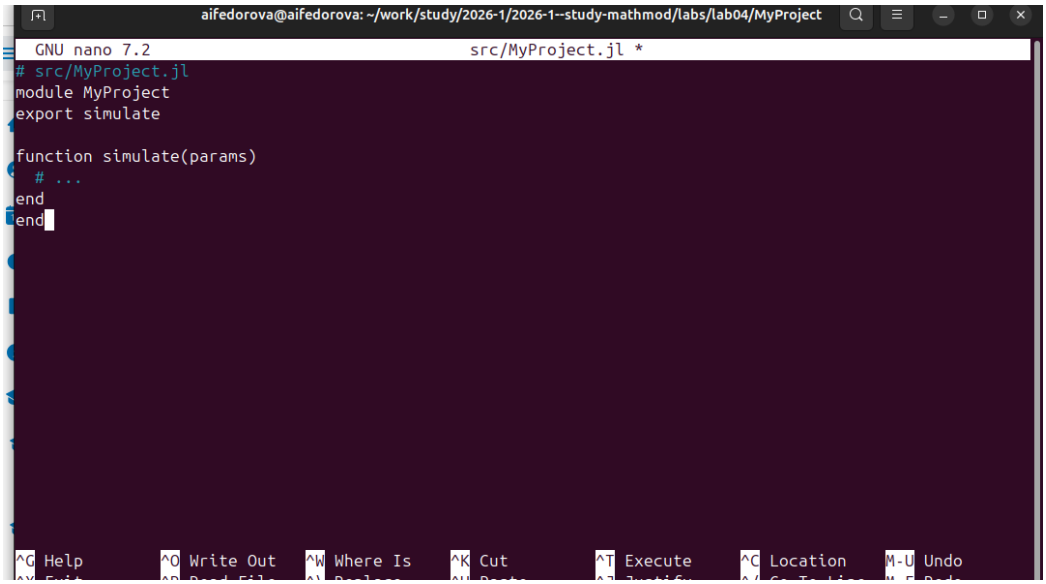

Рисунок 4.7: Скрипт PkgTemplates.jl для создания нового проекта с DrWatsonPlugin

5 Выводы

В ходе лабораторной работы были освоены основные инструменты пакета DrWatson для организации воспроизводимых научных исследований на Julia:

- инициализация проекта с помощью `initialize_project` и автоматическое создание стандартной структуры директорий;
- функции навигации по проекту (`datadir`, `plotsdir`, `projectdir`), позволяющие использовать переносимые пути независимо от расположения проекта;
- именование файлов по параметрам через `savename` и безопасное сохранение данных через `safesave` без риска перезаписи результатов;
- генерация комбинаций параметров через `dict_list` для пакетных симуляций;
- сбор результатов из нескольких файлов в единый DataFrame через `collect_results`;
- автоматизация рабочего процесса с помощью Makefile и Quarto;
- создание шаблонного проекта через PkgTemplates с плагином DrWatsonPlugin.

В процессе работы выявлены типичные ошибки: использование несуществующей функции `scriptdir` вместо `scriptsdir`, отсутствие определения пользовательских функций в текущей сессии, а также необходимость включать параметры симуляции в сохраняемый словарь для корректного последующего анализа через `collect_results`.

Список литературы